

what is the standard file structure in react native?

There isn't a single **official** React Native folder structure, but there **is** a structure that has become the de facto standard for medium-to-large applications. Since I know you've been building **BudgetMaze** with React Native, TypeScript, and reusable UI components, I'd recommend a feature-first architecture with clear separation of concerns.

1. Small Project

Good for learning or MVPs.

```
my-app/
├─ android/
├─ ios/
├─ src/
│   ├─ components/
│   ├─ screens/
│   ├─ navigation/
│   ├─ assets/
│   ├─ services/
│   ├─ utils/
│   ├─ hooks/
│   ├─ constants/
│   ├─ types/
│   └─ App.tsx
├─ package.json
└─ tsconfig.json
```

Pros

- Simple
- Easy to navigate

Cons

- Doesn't scale well.

2. Medium to Large Project (Recommended)

```
BudgetMaze/
|
├─ android/
├─ ios/
```

```
|
|
|─ src/
|   |
|   |─ app/
|   |   |─ App.tsx
|   |   |─ providers/
|   |   |─ navigation/
|   |   └─ routes/
|   |
|   |─ assets/
|   |   |─ images/
|   |   |─ icons/
|   |   |─ fonts/
|   |   └─ animations/
|   |
|   |─ components/
|   |   |─ atoms/
|   |   |─ molecules/
|   |   |─ organisms/
|   |   └─ templates/
|   |
|   |─ features/
|   |   |─ auth/
|   |   |─ budgets/
|   |   |─ transactions/
|   |   |─ goals/
|   |   |─ analytics/
|   |   |─ notifications/
|   |   └─ settings/
|   |
|   |─ services/
|   |   |─ api/
|   |   |─ storage/
|   |   |─ auth/
|   |   └─ notifications/
|   |
|   |─ hooks/
|   |─ contexts/
|   |─ store/
|   |─ theme/
|   |─ utils/
|   |─ constants/
|   |─ types/
|   |─ config/
|   └─ localization/
|
|─ .env
|─ package.json
└─ tsconfig.json
```

3. Feature Folder Structure

Each feature is self-contained.

```
features/  
|  
├─ auth/  
|   ├── api/  
|   ├── components/  
|   ├── hooks/  
|   ├── screens/  
|   ├── navigation/  
|   ├── services/  
|   ├── types/  
|   ├── validators/  
|   └─ index.ts  
|  
├─ budgets/  
|   ├── api/  
|   ├── components/  
|   ├── hooks/  
|   ├── screens/  
|   ├── services/  
|   ├── types/  
|   └─ index.ts  
|  
└─ transactions/
```

This keeps everything related to a feature together, which makes the project easier to maintain as it grows.

4. Components (Atomic Design)

```
components/  
  
atoms/  
  Button/  
  Input/  
  Text/  
  Avatar/  
  Badge/  
  
molecules/  
  SearchBar/  
  PasswordInput/  
  CardHeader/  
  BudgetCard/
```

```
organisms/  
  TransactionList/  
  BudgetOverview/  
  DashboardHeader/
```

```
templates/  
  AuthLayout/  
  DashboardLayout/
```

Atoms

Smallest reusable components.

```
Button  
Input  
Text  
Checkbox  
Icon  
Avatar  
Loader
```

Molecules

Made from atoms.

```
LoginForm  
SearchBar  
CardHeader  
BudgetSummary
```

Organisms

Large UI sections.

```
Dashboard  
TransactionList  
ProfileHeader  
SettingsMenu
```

5. Navigation

navigation/

index.ts

AuthNavigator.tsx

AppNavigator.tsx

RootNavigator.tsx

BottomTabNavigator.tsx

6. API

services/

api/

 axios.ts

 auth.api.ts

 budget.api.ts

 goal.api.ts

 transaction.api.ts

Example:

api/

 client.ts

 endpoints.ts

 interceptors.ts

7. State Management

store/

auth/

budget/

goal/

transaction/

Whether you use Redux Toolkit, Zustand, Jotai, or another solution, keeping state organized by domain helps maintain clarity.

8. Utilities

utils/

currency.ts

date.ts

validation.ts

formatter.ts

storage.ts

9. Hooks

hooks/

useAuth.ts

useDebounce.ts

useNetwork.ts

useTheme.ts

useModal.ts

10. Theme

theme/

colors.ts

fonts.ts

spacing.ts

radius.ts

typography.ts

theme.ts

Avoid hardcoding values like:

```
padding: 17  
color: "#2980B9"
```

Instead:

```
padding: spacing.md  
color: colors.primary
```

11. Types

types/

auth.ts

budget.ts

goal.ts

transaction.ts

api.ts

12. Assets

assets/

images/

icons/

fonts/

animations/

svg/

Example Screen Structure

```
features/  
└─ auth/  
    │─ api/  
    │─ components/  
    │   │─ LoginForm.tsx  
    │   └─ SocialLogin.tsx  
    │─ hooks/  
    │   └─ useLogin.ts  
    │─ screens/  
    │   │─ LoginScreen.tsx  
    │   │─ RegisterScreen.tsx  
    │   └─ ForgotPasswordScreen.tsx  
    │─ services/  
    │─ types/  
    │─ validators/  
    │   └─ login.validator.ts  
    └─ index.ts
```

What I'd Recommend for BudgetMaze

Given the size and direction of your project, a combination of **feature-first organization** and **Atomic Design** is likely the best fit.

- **Feature-first** (features/auth , features/budgets , features/goals , features/transactions) keeps business logic, screens, and APIs together.
- **Shared UI** belongs in components/ (atoms, molecules, organisms).
- **Cross-cutting concerns** like navigation, theme, API client, hooks, and utilities stay in top-level shared folders.
- **State management** is organized by feature rather than by file type.

This approach scales well for teams, reduces coupling between unrelated features, and makes it easier to add new modules without reorganizing the project. It's also a structure commonly seen in production React Native applications.